
Protein Circuit Topology Plugin

User Manual & Documentation

circuittopology@gmail.com

Abstract

This is the official documentation file for the PyMOL Protein Circuit Topology plugin. The plugin wraps most of the functionality of the ProteinCT framework [1] and exposes it through a PyMOL GUI. For questions, suggestions, or bug reports, please contact us via the email address above. The latest version is available at the [GitHub repository](#).

Circuit Topology in one paragraph. Circuit Topology (CT) [2] describes how pairs of intramolecular residue–residue contacts are arranged along the sequence. The plugin exposes the four relations also returned by ProteinCT’s `local_ct`: **Series (S)** — sequentially arranged, non-overlapping contacts; **Parallel (P)** — one contact nested inside the other; **Inverse parallel (IP)** — inverse nesting; and **Cross (X)** — interleaved or linked contacts. A *contact map* is the set of residue pairs within a distance cutoff; the plugin renders it as a residue $\times$ residue matrix or as an arc (“circuit”) diagram above the sequence. In a *relations matrix*, every contact-pair cell is coloured by its CT relation.

A precise specification of what the analysis does and does not do — which atoms are included, how contacts are detected, and two known limitations inherited from ProteinCT (insertion codes and chain breaks) — is given in `documentation/METHODS.md` in the repository.

Contents

1	Installation Guide	2
1.1	Downloading and Installing PyMOL	2
1.2	Mandatory Dependencies	2
1.3	Plugin Activation Steps	2
1.4	What the Automatic Installer Does	4
2	User Guide	5
2.1	Local Circuit Topology (LCT) Protocol	5
2.2	Single-File Analysis Protocol	9
2.2.1	Visualizing Topology on the 3D Structure in PyMOL	11
2.2.2	CT Folding Score	13
2.3	Multi-File Analysis Protocol	15

1 Installation Guide

1.1 Downloading and Installing PyMOL

Download the PyMOL 3.1.6.1 installer for your platform (Windows / Linux / macOS) from <https://www.pymol.org/>. Previous versions are available at:

<https://storage.googleapis.com/pymol-storage/installers/index.html>

During setup, install PyMOL for the current user only by selecting **Just Me (recommended)**. This ensures the plugin’s automatic dependency installer has the necessary write permissions. If PyMOL is installed for all users, the installer needs administrator or root privileges; the plugin detects this, explains it, and prints the command to run yourself.

When PyMOL prompts you to register file extensions, select **Register Recommended**. The plugin primarily works with `.pdb`, `.cif`, and `.xtc` files.

1.2 Mandatory Dependencies

Table 1: Required Python and PyMOL versions. These are the versions the released outputs were produced with.

Dependency	Version	Dependency	Version
Python	3.10.18	NumPy	1.26.4
PyQt	5.15.11	Pandas	2.3.3
Qt-Main	5.15.15	Matplotlib	3.9.1
PyQt5-sip	12.17.0	PyMOL	3.1.6.1
Biopython	1.85	PyMOL-Bundle	3.1.6.1

Only four of these are the plugin’s own requirements — NumPy, Pandas, Matplotlib and Biopython — and they are declared in `requirements.yml`. Everything else ships with the PyMOL bundle. On a stock PyMOL 3.1.6.1 installation only `pandas` and `matplotlib` are actually missing, so those are the only two packages the automatic installer normally has to fetch.

1.3 Plugin Activation Steps

1. Download the latest plugin release `.zip` directly from GitHub:

[protein-circuit-topology-pymol-plugin.zip](#)

For a specific tagged version, browse the [Releases](#) page.

2. Open PyMOL and navigate to **Plugin** → **Plugin Manager**.
3. Switch to the **Install New Plugin** tab and click **Choose file...**, then select the `.zip` you just downloaded.

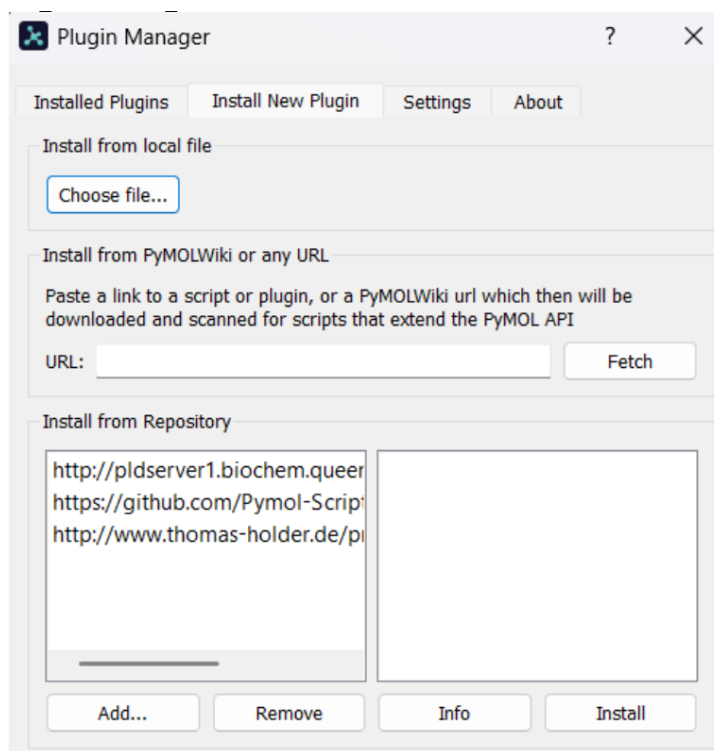


Figure 1: PyMOL's Plugin Manager with the **Install New Plugin** tab open. Use the **Choose file...** button under *Install from local file* to select the downloaded .zip.

4. Wait for the plugin to install and configure its dependencies. After installation, launch the plugin from **Plugin** → **Protein Circuit Topology Plugin**.

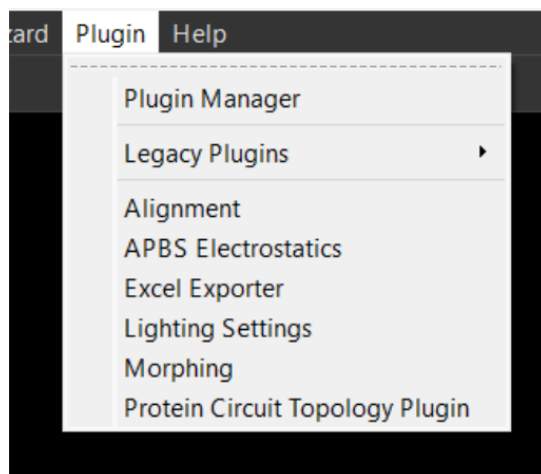


Figure 2: The plugin appears under the PyMOL **Plugin** menu after installation.

Important Installation Notes

- **Initial setup:** the first time the plugin is loaded, PyMOL may need extra time to download and configure the missing dependencies. Progress is printed to the PyMOL command line.
- **Restart prompt:** if the packages install correctly but Python cannot import them in the already-running session, the plugin says so and asks you to restart PyMOL. Nothing is lost — reopen PyMOL and the plugin is ready.
- **Nothing is replaced:** the installer only *adds* packages. PyMOL and Python are pinned to the versions you already have, so the solver cannot swap them out while resolving the new dependencies.

1.4 What the Automatic Installer Does

The installer works entirely through the conda environment that PyMOL itself runs in. It never creates a new environment and never touches any other environment on the system. In outline it

1. locates the conda belonging to the running PyMOL (CONDA_EXE first, then the usual paths under `sys.prefix`, then `conda` on `PATH`);
2. works out which of the four requirements are missing; and
3. installs only those, with the interpreter and PyMOL pinned:

```
conda install --yes --freeze-installed --prefix <PyMOL's sys.prefix>
python==<installed> pymol==<installed>
conda-forge::<each missing package>
```

The pins matter. Without them conda satisfies the new dependencies by replacing PyMOL itself, which leaves an installation that no longer starts.

Manual Installation

If you would rather install the dependencies yourself, run the equivalent command before loading the plugin. Find the prefix and the versions to pin with

```
python -c "import sys, pymol; print(sys.prefix,
sys.version.split()[0], pymol.cmd.get_version()[0])"
```

and then

```
conda install --yes --freeze-installed --prefix <prefix>
python==<version> pymol==<version>
"conda-forge::numpy">=1.23,<2" "conda-forge::pandas">=2.0,<3"
"conda-forge::matplotlib">=3.7,<4" "conda-forge::biopython">=1.80,<2"
```

2 User Guide

The GUI opens as a single window titled *Circuit Topology Tool* with three tabs: **Local Circuit Topology**, **Single-File Analysis** and **Multi-File Analysis**. Small grey *i* buttons throughout the interface explain individual controls on hover or click.

Non-polymer atoms are handled for you

Circuit Topology analyses polymer atoms only. Waters, ions, ligands and other non-polymer atoms are excluded automatically at analysis time, and nucleic-acid and UNK residues are dropped when the chain is parsed. Nothing is removed from your loaded object, and no cleanup step is required before running an analysis. When you pick an object, the plugin prints to the PyMOL command line exactly what it will ignore, for example:

Excluded 118 non-polymer atoms (58 HOH, 2 ZN) from analysis; 1bcs is unchanged.

2.1 Local Circuit Topology (LCT) Protocol

The Local Circuit Topology tab analyses the topological relationships of one targeted residue against the rest of the structure.

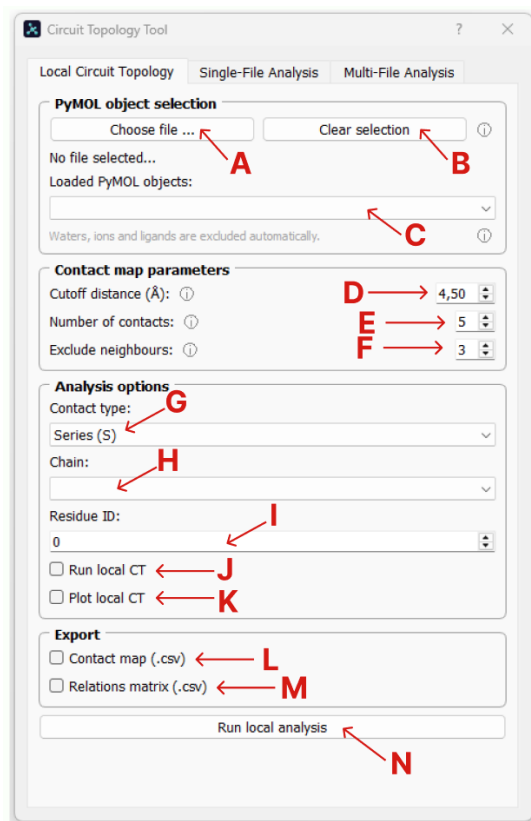


Figure 3: The Local Circuit Topology tab, with the controls used in this protocol labelled A–N: object selection (A–C), contact-map parameters (D) cutoff distance, (E) number of contacts, (F) exclude neighbours; analysis options (G) contact type, (H) chain, (I) residue ID, (J) run local CT, (K) plot local CT; exports (L) contact map, (M) relations matrix; and the run button (N).

To narrow the analysis to a specific region, select a chain (**H**) and a residue ID (**I**), then choose the contact type (**G**) of interest. With **Run local CT (J)** ticked the plugin computes, for every contact that begins or ends at the selected residue, how it relates topologically to every other contact in the structure. Results can be exported as `.csv` files via (**L**) and (**M**).

LCT Step-by-Step Guide

1. **Load a structure:** click **Choose file ... (A)** to load a `.pdb` or `.cif` file, or pick an already-loaded PyMOL object from the **Loaded PyMOL objects** dropdown (**C**). **Clear selection (B)** unloads the current file or object from PyMOL again.
2. **Define contact parameters (D–F):**
 - **Cutoff distance (Å) (D)** — maximum atom–atom distance counted as a contact. Range 0.1–20.0, default 4.5.
 - **Number of contacts (E)** — minimum number of atom–atom contacts required before a residue–residue pair counts as a contact. Range 5–45, default 5.
 - **Exclude neighbours (F)** — discards trivial contacts between residues close in sequence. Range 1–10, default 3.
3. **Target a region:** pick the **Chain (H)** and the **Residue ID (I)**. The residue spinner is bounded by the residues actually present in the chosen chain, and it is repopulated whenever you change object or chain.
4. **Choose Contact Type (G):** **Series (S)**, **Parallel (P)**, **Inverse parallel (IP)** or **Cross (X)**.
5. **Output options:**
 - Tick **Run local CT (J)** to compute the local topology counts, which are printed to the PyMOL command line.
 - Tick **Plot local CT (K)** to draw the plots shown below.
 - Tick **Contact map (.csv) (L)** or **Relations matrix (.csv) (M)**. Ticking either one reveals a **Choose output directory ...** button below the checkboxes; pick a destination before running.
6. **Run:** click **Run local analysis (N)**.

When nothing is highlighted

Not every combination of chain, residue and contact type has anything to show. If the selected residue takes part in contacts but none of them stand in the requested relation, the plugin says so on the command line, and suggests trying another contact type or residue. If the residue takes part in no contacts at all, that is reported too — a common outcome when the cutoff distance is small or **Exclude neighbours** is large.

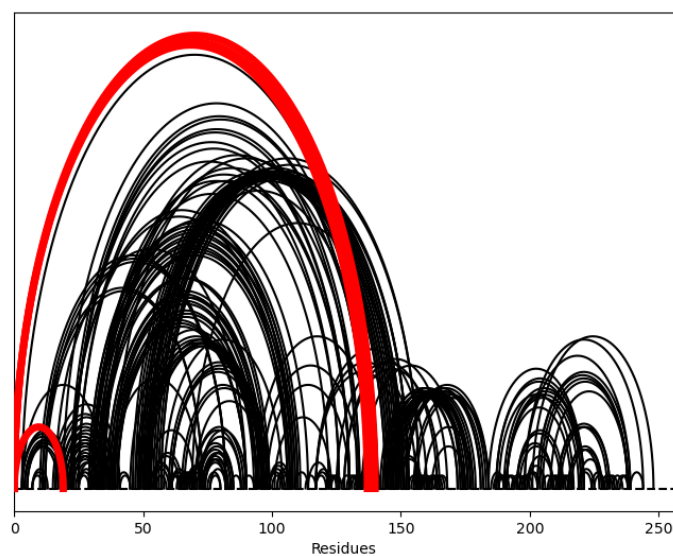


Figure 4: Local CT plot, step 1: contacts that begin or end at the selected residue are highlighted in red over the background contact map.

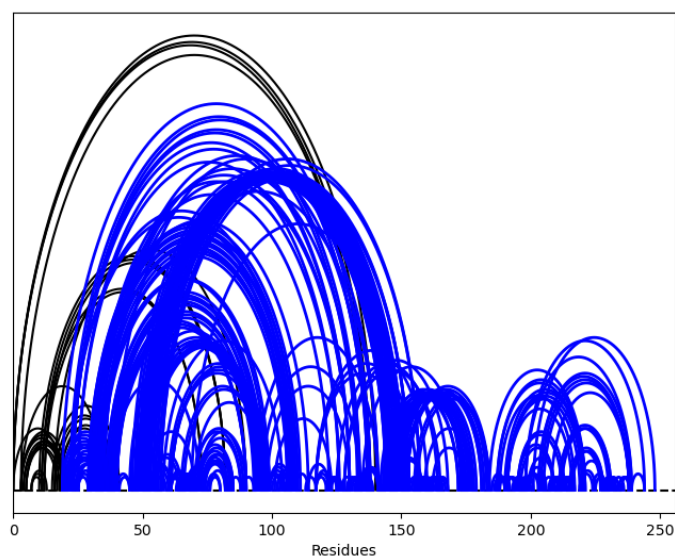


Figure 5: Local CT plot, step 2: contacts whose CT relation to the red ones matches the contact type selected in dropdown (**G**) are drawn in blue.

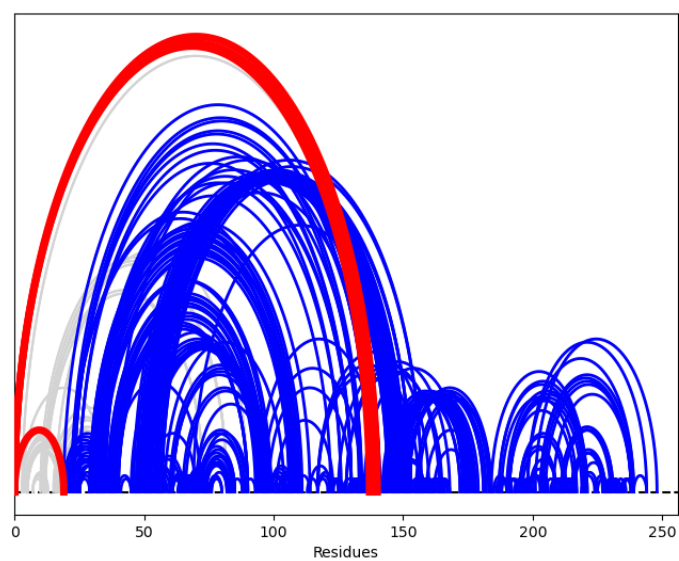


Figure 6: Combined local CT plot showing both the red (residue-anchored) and blue (relation-selected) curves, allowing visual identification of local topological relationships around the selected residue.

2.2 Single-File Analysis Protocol

The Single-File Analysis tab gives a global topological overview of one structure: a relations matrix, the corresponding circuit (arc) representation, an optional scalar **Folding Score** per chain, and direct recolouring of the structure in the PyMOL viewer.

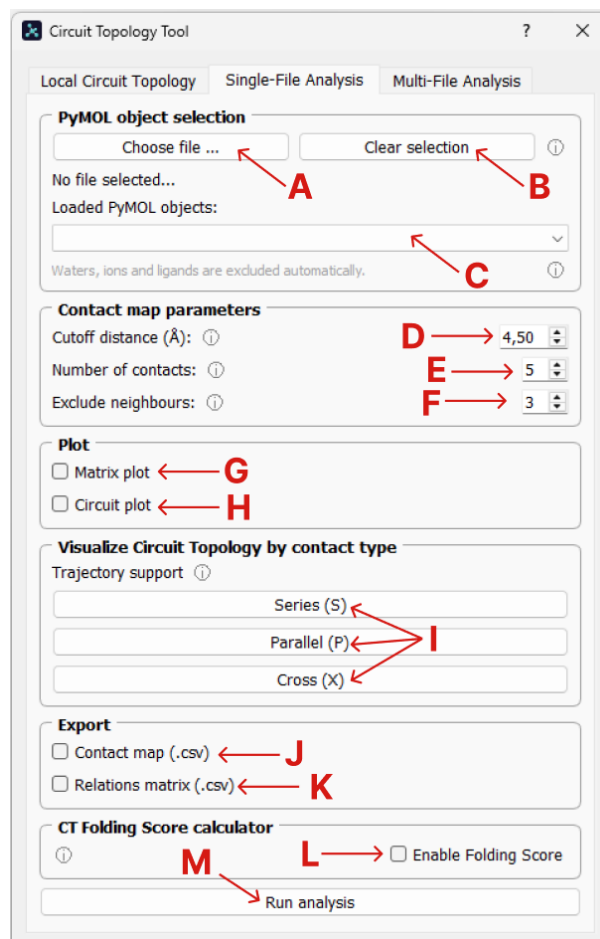


Figure 7: The Single-File Analysis tab, labelled A–M: object selection (A–C), contact-map parameters (D–F), plot toggles (G) matrix and (H) circuit, the *Visualize Circuit Topology by contact type* buttons (I) (Series, Parallel, Cross), export checkboxes (J–K), the Folding Score toggle (L), and the run button (M).

The **Matrix plot** (G) renders the relations matrix, in which each cell shows the CT relation between two contacts. For *single-chain* structures the matrix uses the standard P / S / X categories together with their inverse variants. For *multi-chain* structures the plugin adds the further categories visible in the legend of Figure 9. The **Circuit plot** (H) draws every contact as an arc above the residue index; for multi-chain inputs each chain is rendered in a different colour (Figure 8).

*Note: when both **Matrix plot** (G) and **Circuit plot** (H) are ticked, the plugin opens two separate windows — one per plot.*

Single-File Analysis Guide

1. **Load the protein (A–C):** load a .pdb or .cif file, or pick an existing PyMOL object from the dropdown.

2. **Define contact parameters (D–F):** cutoff distance, number of contacts and excluded neighbours — the same parameters, ranges and defaults as on the LCT tab.
3. **Select plots:** tick **Matrix plot (G)** for the relations matrix and **Circuit plot (H)** for the arc diagram.
4. **(Optional) Folding Score:** tick **Enable Folding Score (L)** to compute a per-chain CT Folding Score (Section 2.2.2).
5. **Export data:** tick **Contact map (.csv) (J)** and/or **Relations matrix (.csv) (K)**, then choose an output directory in the widgets that appear.
6. **Run:** click **Run analysis (M)**.

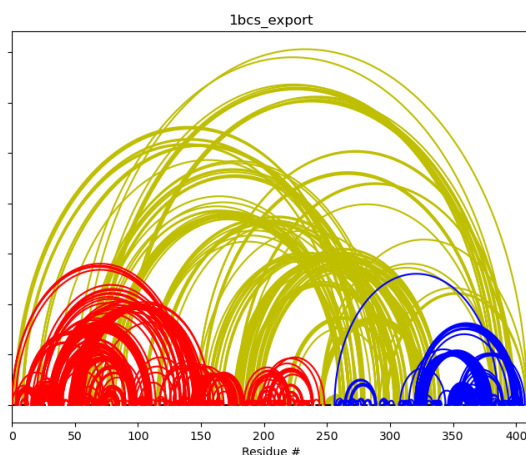


Figure 8: Circuit (arc) plot produced by the Single-File Analysis tab for the multi-chain object `1bcs_export`. Each chain is drawn in a distinct colour; arcs spanning chains highlight inter-chain contacts.

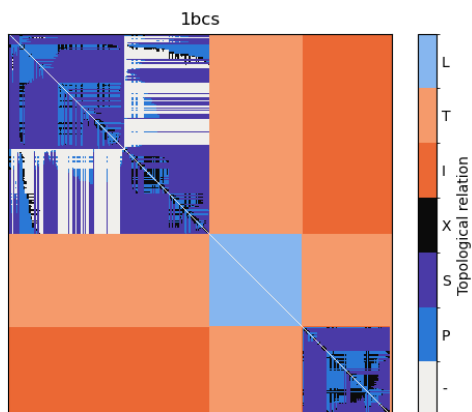


Figure 9: Relations-matrix plot for the same multi-chain object. The legend uses the multi-chain contact categories, which appear only for multi-chain proteins. For single-chain inputs the legend reduces to the standard CT classes.

2.2.1 Visualizing Topology on the 3D Structure in PyMOL

The **Visualize Circuit Topology by contact type** panel (Figure 7, button row (I)) recolours the structure in the PyMOL viewer according to how densely each residue participates in a given CT relation. Pressing **Series (S)**, **Parallel (P)** or **Cross (X)** runs the analysis for each chain of the selected object and then, per chain, calls

```
get_topology_vector(mat, index, topology_type, numbering)
color_by_topology(molecule_name, topology_vector, numbering,
                 topology_type)
```

`get_topology_vector` returns one value per residue that counts how many relations of the selected class that residue takes part in. The value is aligned element-for-element with the residue numbering.

`color_by_topology` writes the counts into the B-factor column and colours the residues by them.

Turning counts into colours

Residues that take part in *no* relation of the selected type are white. Every other residue is placed in one of five levels, from pale tint at lowest to full saturation at highest.

The level boundaries are the *quantiles* of the non-zero counts. This accounts for the heavily right-skewed nature of the residue counts. Spread evenly from zero to the maximum, a few outliers would consume the whole scale — in chain A of **1bcs** the Series counts reach 1900, so a residue taking part in 50 relations would sit 2.6 % along a white-to-colour ramp and render as white, indistinguishable from a residue taking part in none. Quantile levels guarantee that every shade is used, so low activity reads as low activity rather than as absence.

Trajectories

The buttons act on whichever object is selected in the dropdown. If that object is a trajectory with more than one state, *every* state is coloured: the topology analysis runs for each frame and the frames are merged into a single multi-state object named `<object>_topo` that you can scrub through with PyMOL's frame controls. Every frame is measured before any frame is coloured, so the whole trajectory shares one scale and one legend — otherwise a residue would change colour between frames merely because that frame's distribution shifted, which is exactly the sort of thing an animation makes look real. For long trajectories this takes a while and uses significant memory. Non-polymer atoms are excluded before the frames are split, so solvent never enters the calculation.

Warnings you may see. Two limitations inherited from ProteinCT are reported at runtime instead of silent ignore. Contact detection matches on the residue number alone, so where **insertion codes** are in use the atoms of an inserted residue are attributed to the first residue with that number. Neighbour exclusion is applied by position in the residue list rather than by residue number, so residues flanking a **chain break** are treated as sequence neighbours. Both are explained in [documentation/METHODS.md](#). A representative log for the three-chain structure **1bcs** looks like this:

```
INFO - Chains found:  ['A', 'B', 'C']  
WARNING - 1bcs_topo_A: 4 residue number(s) occur more than once, i.e. insertion  
codes are in use (23, 58, 76, 112).  
WARNING - 1bcs_topo_A: 2 chain break(s) in the residue numbering (11->14,  
134->137).  
INFO - Analysing 1bcs_topo, chain A ...  
WARNING - No contacts found for chain C. Skipping visualization...  
INFO - Coloured '1bcs_topo' by S topology. '1bcs' keeps its original B-factors.
```

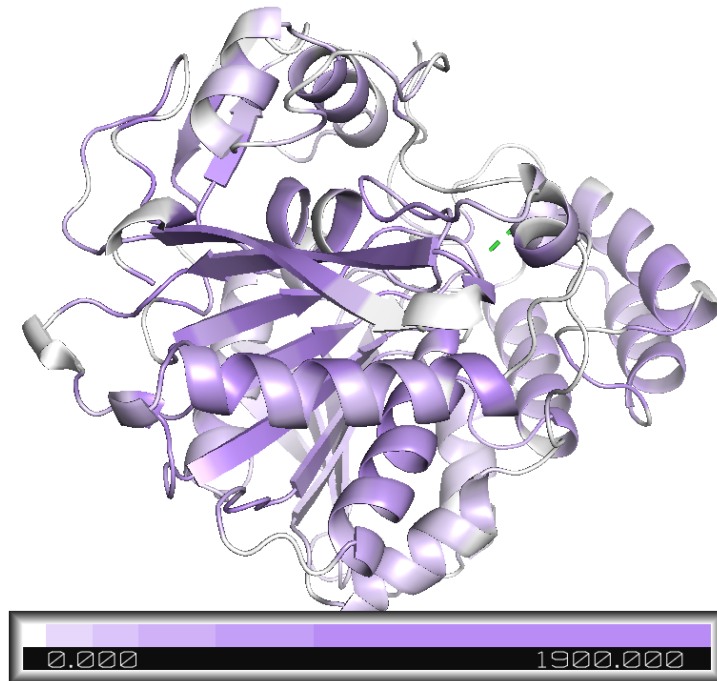


Figure 10: Contact type **Series (S)** visualized for 1bcs. Chain C yielded no contacts and was skipped.

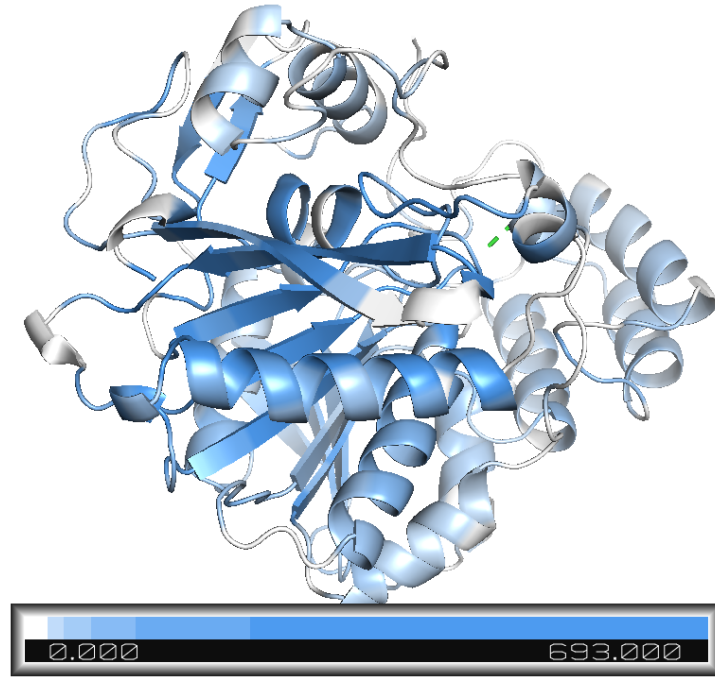


Figure 11: Contact type **Parallel (P)** visualized for 1bcs.



Figure 12: Contact type **Cross (X)** visualized for 1bcs. Comparing the three figures shows which regions are densely involved in each relation class.

2.2.2 CT Folding Score

Ticking **Enable Folding Score (L)** reduces the contact map and relations matrix to a single number per chain: the sum of the mean per-residue Parallel, Series and Cross densities, computed by `get_folding_score`. Larger values indicate a contact network in which the average residue participates in more topological relations. The score builds on

the circuit-topology-based fold analysis framework of Akulov *et al.* [3] and gives a quick way to compare conformations without inspecting full plots.

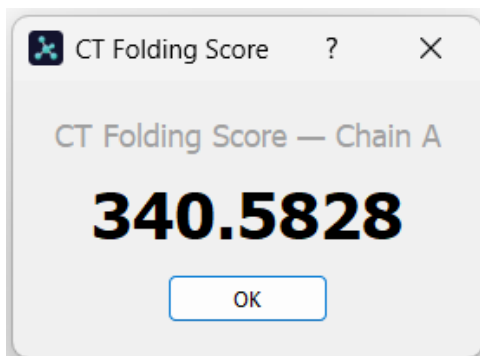


Figure 13: CT Folding Score for protein 1a6n.

2.3 Multi-File Analysis Protocol

The Multi-File Analysis tab batch-processes a set of structures: either a directory of .pdb/.cif files, or a trajectory loaded as a topology .pdb plus an .xtc trajectory file. Because this tab is taller than the window, it scrolls.

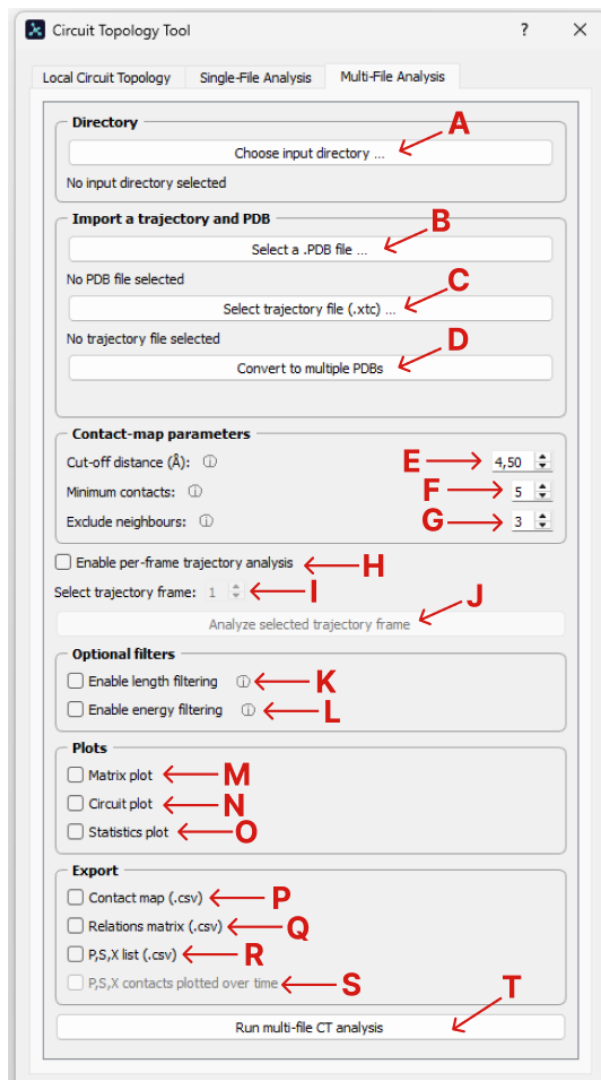


Figure 14: The Multi-File Analysis tab, labelled A–T: directory input (A), trajectory import (B–C), trajectory-to-PDB conversion (D), contact-map parameters (E–G), per-frame analysis (H–J), optional filters (K) length and (L) energy, plot toggles (M) matrix, (N) circuit, (O) statistics, export checkboxes (P–S), and the run button (T).

Input Modes

Two input modes are supported. Either

- point at a **directory** of structures via **Choose input directory ...** (A), or
- import a topology and trajectory pair: **Select a .PDB file ...** (B) and **Select trajectory file (.xtc) ...** (C).

With a topology and trajectory loaded, **Convert to multiple PDBs** (D) writes each frame to its own .pdb file, turning the trajectory into a directory you can then point at

for a regular multi-file run. This is optional: a loaded trajectory can be analysed directly, frame by frame, without exporting anything.

Per-Frame Analysis

Ticking **Enable per-frame trajectory analysis (H)** enables the **Select trajectory frame** spinner (**I**) and the **Analyze selected trajectory frame** button (**J**), which processes only the chosen frame. Both stay greyed out until the checkbox is ticked. As an alternative, the imported trajectory object can be selected on the Single-File Analysis tab and stepped through with PyMOL's own frame controls.

Optional Filters

- **Enable length filtering (K)**: keeps only contacts whose sequence separation matches the chosen comparator — **equality (=)**, **less than (<)** or **greater than (>)** — against a **Distance** threshold. Ticking the box reveals the threshold spinner and the comparator dropdown.
- **Enable energy filtering (L)**: retains only **Attractive / stabilising (+)** or only **Repulsive / destabilising (-)** contacts. Ticking the box reveals the mode dropdown.

Filter caveats

Both filters were designed for single-chain proteins and are skipped automatically when the input contains multiple chains — the plugin reports which file and which filter it skipped. They also have no effect during *per-frame trajectory analysis*, which runs the unfiltered pipeline on the chosen frame. Aggressive length-filter thresholds can produce empty contact maps.

Plots and Exports

Three plot checkboxes are available:

- **Matrix plot (M)** and **Circuit plot (N)** — the same outputs as the Single-File tab, generated for every input file.
- **Statistics plot (O)** — a two-panel figure: the fraction of entangled (Parallel and Cross) contacts against distance from the matrix diagonal, beside a pie chart of the Parallel, Series and Cross proportions.

Three export checkboxes write CSV files to the chosen output directory: the residue contact map (**P**), the relations matrix (**Q**), and the per-residue P, S and X counts (**R**). Ticking the P, S, X list enables the fourth checkbox, **P,S,X contacts plotted over time (S)**, which plots the three counts across the input set. That is only meaningful for a genuine trajectory, i.e. when each input file is a consecutive frame of the same structure. As on the other tabs, the output-directory widgets appear as soon as any export is ticked.

Multiple-File Analysis Guide

1. **Load the input:** pick an input directory (**A**) of `.pdb/` `.cif` files, or load a topology and trajectory pair (**B–C**).
2. **(Optional) Convert the trajectory to PDBs (D)** if you want the individual frames as standalone files.
3. **Define contact parameters (E–G):** cut-off distance, minimum contacts, exclude neighbours.
4. **(Optional) Per-frame analysis (H–I):** tick **Enable per-frame trajectory analysis** and choose the frame index.
5. **(Optional) Apply filters (K–L):** enable length and/or energy filtering, subject to the caveats above.
6. **Pick plots (M–O):** matrix, circuit, statistics.
7. **Pick exports (P–R):** contact map, relations matrix, P, S, X list, and optionally (**S**) for the time series.
8. **Run:** click **Run multi-file CT analysis (T)** to process the whole input set, or **Analyze selected trajectory frame (J)** to process just the chosen frame.

References

- [1] Moes, D., Banijamali, E., Sheikhhassani, V., Scalvini, B., Woodard, J., Mashaghi, A. (2022). *ProteinCT: An implementation of the protein circuit topology framework*. MethodsX, open access (PII S2215016122002400). Source code: https://github.com/circuittopology/circuit_topology.
- [2] Mashaghi, A., Van Wijk, R.J., Tans, S.J. (2014). *Circuit topology of proteins and nucleic acids*. Structure 22(9), 1227–1237. doi:10.1016/j.str.2014.06.015.
- [3] Akulov, V., Panizo, A.J., Estébanez-Perpiñá, E., van Noort, J., Mashaghi, A. (2024). *Phosphorylation regulated conformational diversity and topological dynamics of an intrinsically disordered nuclear receptor*. bioRxiv 2024.09.21.614239, Cold Spring Harbor Laboratory. doi:10.1101/2024.09.21.614239.